

HCM360 HRIS

Access and Security

Roles, permissions, authentication, and data privacy in HCM360

Version 1.0.0 · April 2026

1. Security Architecture

HCM360 applies security at multiple layers. Authentication identifies the caller; authorization decides what they can see and do; data privacy filters what appears in the response. The whole stack is reinforced by audit logging so every mutation is traceable.

1.1 Layered Controls

Layer	Mechanism	Scope
Authentication	Session login + optional SSO JWT	Identifies the user
Page Access	core.role_page_access + core.user_page_access	URL-level allow/deny
Feature Access	core.role_feature_access + core.user_feature_access	Button/action level
Modification Access	core.mod_permissions	Module-level write capability
Field Privacy	core.field_privacy_rules	Per-field VISIBLE/MASKED/HIDDEN
Audit Log	audit_logs.change_log triggers	Immutable change history

2. Authentication

v1.0 ships with a dropdown-based login (POC mode) that selects a user from core.users. The session is cookie-based and signed with SECRET_KEY. Production deployments should replace this with password + MFA or SSO.

2.1 Login Flow

- 1 GET /login renders a user selector from core.users WHERE is_active = TRUE
- 2 POST /login/select writes session["user_id"] = <id>
- 3 before_request hook re-derives session["role_code"] and session["employee_id"] on every request
- 4 Unauthenticated requests (except /login, /static, /inquiry) are redirected to /login

2.2 SSO Token Support

A signed JWT can be passed as ?sso_token=... to auto-login. The username claim is matched against email local-part or display_name (lowercased, whitespace removed). Production should enable signature verification.

2.3 Session Hardening

- SECRET_KEY must be long, random, and rotated periodically
- Set SESSION_COOKIE_SECURE=True and SESSION_COOKIE_HTTPONLY=True behind TLS
- Set SESSION_COOKIE_SAMESITE="Strict" to prevent CSRF on top-level nav
- Use SESSION_REFRESH_EACH_REQUEST=True for sliding expiration

3. Built-in Roles

Code	Name	Summary
SUPER_ADMIN	Super Administrator	Full access to every page + feature; bypasses all checks in code
HR_ADMIN	HR Administrator	Full HR operations; restricted from system admin (access matrix, demo data)
MANAGER	People Manager	Team management only; no admin access
EXECUTIVE	Executive	Read-only dashboards and analytics; no admin access
EMPLOYEE	Employee	Self-service only (own DTR, leave, payslips, profile)

Additional custom roles can be created at `/admin/access-matrix` → Create Role. Each role starts with no access and grants are added explicitly.

4. Page-Level Access

Every UI route is registered in `core.page_registry` with a path, title, module, and navigation metadata. Access is granted per role in `core.role_page_access` or per user in `core.user_page_access`. User-level grants override role-level.

4.1 Access Check Algorithm

- 1 If `role_code = SUPER_ADMIN` → return TRUE (bypass)
- 2 Look up page by path (LIKE match so `/employees/` matches `/employees/123`)
- 3 Check `core.user_page_access` for this `user_id` — if found, return `can_access` value
- 4 Check `core.role_page_access` for this `role_code` — return `can_access` or FALSE if no row

4.2 Key Page Paths

Path	Access
/admin/access-matrix	SUPER_ADMIN only (explicitly denied to HR_ADMIN)
/admin/privacy	SUPER_ADMIN only
/admin/demo-data	SUPER_ADMIN only
/admin/themes, /admin/reference, /admin/users	SUPER_ADMIN + HR_ADMIN
/employees/, /attendance/, /leave/, /payroll/	SUPER_ADMIN + HR_ADMIN + MANAGER + EXECUTIVE
/me/*, /team/*	All roles (except MANAGER blocked from /team/* if no direct reports)
/workforce-planning/	SUPER_ADMIN + HR_ADMIN + EXECUTIVE
/	All active roles

5. Feature-Level Access (Actions)

Fine-grained actions live in core.feature_registry with feature_type=ACTION and an action_type (CREATE, EDIT, DELETE, APPROVE, EXPORT, PRINT, OVERRIDE, ADJUST, TOGGLE).

Templates render action buttons conditionally on feature access.

5.1 Example Action Features

Feature Code	Purpose
ATT_DTR_EXPORT	Download DTR as Excel/CSV
ATT_DTR_OVERRIDE	Manually edit a DTR record (creates audit trail)
ATT_OT_APPROVE	Approve an overtime request
LV_REQUEST_CREATE	File a leave request
LV_BALANCE_ADJUST	Manually adjust a leave balance
PAY_RUN_CREATE	Create a new payroll run
PAY_RUN_APPROVE	Approve a computed payroll run
PAY_SLIP_PRINT	Print/download payslip PDF
ADMIN_ACCESS_EDIT	Edit access matrix grants
EMP_PRIVACY_ADMIN	Manage field privacy rules

6. Modification Access

core.mod_permissions separates "can view" from "can modify" at the module level. This lets an EXECUTIVE see Payroll dashboards without ability to change anything. Configured at /admin/modification-access.

Module	Default Allowed
core, employees	SUPER_ADMIN, HR_ADMIN
attendance	SUPER_ADMIN, HR_ADMIN, MANAGER (own team only)
leave_mgmt	SUPER_ADMIN, HR_ADMIN
payroll	SUPER_ADMIN, HR_ADMIN
all other modules	SUPER_ADMIN, HR_ADMIN

7. Field-Level Privacy (RA 10173)

Sensitive personally identifiable information (PII) is masked according to rules in core.field_privacy_rules. The privacy service applies rules to every response before template rendering.

7.1 Visibility States

State	Behavior
VISIBLE	Field passes through unchanged
MASKED	Value replaced with mask_char (default ***); field label still visible
HIDDEN	Field key removed from dict — does not appear in HTML at all

7.2 Default Privacy Matrix

Section	Field	EMPLOYEE	MANAGER
employee	date_of_birth	MASKED	MASKED
employee	mobile_no	VISIBLE (own)	VISIBLE
employee	basic_salary	HIDDEN	HIDDEN
employee	daily_rate	HIDDEN	HIDDEN
employee	salary_grade	HIDDEN	HIDDEN
government_id	id_number	MASKED	VISIBLE
bank_account	account_number	MASKED	MASKED
dependent	date_of_birth	MASKED	MASKED

HR_ADMIN and SUPER_ADMIN bypass all masking by default. Administrators change the matrix at /admin/privacy.

7.3 Self-Record Exception

When is_own_record=True is passed to apply_privacy (set by the ESS route when an employee views their own profile), MASKED fields become VISIBLE but HIDDEN fields stay HIDDEN. This lets an employee see their own mobile number and date of birth but never their salary or rate fields.

8. Audit Logging

The audit_logs schema captures every mutation on business tables. Triggers fire on INSERT, UPDATE, and DELETE and write a row with the before-image, after-image, user, company, and timestamp.

Table	Records
audit_logs.change_log	Every mutation (table, record_id, action, before_json, after_json, user_id, company_id, occurred_at)
audit_logs.access_log	403 denials, failed logins, sensitive exports

Application sets the current user and company with SET LOCAL app.current_user_id / app.current_company_id at the start of each request, which the trigger reads via current_setting().

9. Data Encryption

- In transit: TLS required at the reverse proxy (nginx / Caddy). The Flask app itself is HTTP-only internally
- At rest: PostgreSQL disk encryption via LUKS or cloud provider (AWS EBS, GCP PD)
- Passwords: password_hash column uses werkzeug bcrypt when password login is enabled

- Secrets: .env never committed to git; production uses Docker secrets or vault integration

10. Compliance Coverage

Regulation	Mechanism
RA 10173 (Data Privacy Act)	Field-level masking, audit logs, access controls, consent management via form_fields
DOLE Labor Code	Payroll, leave, OT, holiday pay per statutory rules
BIR	10-year payroll retention, 2316 export, 1601-C generation
SSS / PhilHealth / Pag-IBIG	R3, RF1, MCRF, PRN generation; employer ID per company
CSC (for gov't entities)	Qualification standards, eligibility tracking

11. Security Best Practices for Administrators

- Rotate SECRET_KEY and database password after initial deployment
- Disable all demo users (superadmin, hradmin, manager, etc.) and create real accounts
- Review field privacy rules after every organizational change
- Enable MFA once password login is activated (fields exist in core.users)
- Monitor audit_logs.access_log for failed-login spikes
- Keep Docker images patched — rebuild quarterly with latest base image
- Restrict DB port 5440 to localhost only; never expose externally
- Verify daily backups are readable via scripts/restore_v1.sh on a test environment

12. Incident Response

- 1 Isolate — take the affected container offline: docker compose stop hris_web
- 2 Preserve — snapshot the DB: docker exec hris_db pg_dump ... and copy audit_logs
- 3 Investigate — query audit_logs.change_log and access_log for suspect activity
- 4 Remediate — reset SECRET_KEY, rotate passwords, patch vulnerabilities
- 5 Restore — from last known good backup if data integrity is in question
- 6 Report — notify the data protection officer and file NPC notification if PII is involved (RA 10173 72-hour rule)